Understanding FlashAttention

How IO-Aware Attention Makes Transformers Faster Without Approximating Attention

---

A technical handbook on exact tiled attention: GPU memory traffic, online softmax, forward and backward passes, IO complexity, the evolution from FlashAttention-1 through FlashAttention-4, and current framework behavior.

@techNmak

How to read this handbook

FlashAttention is easiest to understand when three different questions are kept separate:

1. **What mathematical function is being computed?** For dense attention, the target remains ordinary scaled dot-product attention.
2. **How much arithmetic does that function require?** Dense all-pairs query-key scoring remains quadratic in sequence length.
3. **How does the implementation move data through the GPU memory hierarchy?** This is where FlashAttention changes the algorithmic execution dramatically.[4]

The central lesson is that wall-clock speed is not determined by FLOP count alone. An algorithm can perform essentially the same mathematical work, and sometimes even recompute intermediate values, yet run faster because it moves far less data to and from high-bandwidth memory.

> **Core distinction.** Dense FlashAttention is an *exact attention algorithm*: it does not replace softmax attention with a low-rank, sparse, kernelized, or approximate formula. "Exact" refers to the mathematical attention computation. Floating-point kernels can still differ by small rounding effects because operations are reordered.[4, 10, 13]

Notation

For one attention head, let

$$Q \in \mathbb{R}^{N_q \times d}, \qquad K \in \mathbb{R}^{N_k \times d}, \qquad V \in \mathbb{R}^{N_k \times d_v}.$$

For self-attention, typically $N_q = N_k = N$. The scaled score matrix is

$$S = \frac{QK^\top}{\sqrt{d}} + B,$$

where B represents an optional additive mask or bias, and

$$P = \mathrm{softmax}_{\mathrm{row}}(S), \qquad O = PV.$$

We use **HBM** for large off-chip high-bandwidth GPU memory and **on-chip memory** as a broad teaching term for much smaller, faster storage such as registers and shared memory/SRAM. Exact hardware details vary by GPU generation.

Contents

1	What FlashAttention actually optimizes	5
2	The attention equation is not the implementation	6
3	GPU memory hierarchy and why IO matters	7
4	Why materializing S and P is expensive	8
5	Dense arithmetic is still quadratic	9
6	Memory-efficient exact attention predates FlashAttention	10
7	Tiling queries, keys, and values	11
8	Softmax is the difficult part of streaming	12
9	Online softmax from first principles	13
10	The blockwise merge recurrence	14
11	A complete numerical example	15
12	FlashAttention forward pass	16
13	Why dense FlashAttention is exact	17
14	IO complexity: what the theorem actually says	18
15	Memory complexity: linear auxiliary state, not linear compute	19
16	Causal masking and tile skipping	20
17	Backward pass: recompute instead of store	21
18	Why more FLOPs can still be faster	22
19	MHA, MQA, and GQA compatibility	23

Contents continued

20	Variable lengths, local attention, and dropout	24
21	FlashAttention-2: what changed	25
22	FA2 parallelism across sequence tiles	26
23	FA2 work partitioning and non-matmul FLOPs	27
24	FlashAttention-3: the Hopper generation	28
25	FA3 asynchrony: overlap data movement, GEMM, and softmax	29
26	FA3 FP8: performance without pretending precision is free	30
27	FlashAttention-4: the Blackwell generation	31
28	FA4 and asymmetric hardware scaling	32
29	FA4 implementation and current status	33
30	FlashAttention-1 through -4 compared	34
31	PyTorch scaled-dot-product attention today	35
32	Exactness is not bitwise identity	36
33	FlashAttention vs. PagedAttention	37
34	FlashAttention vs. sparse and linear attention	38
35	Training, prefill, and decode are different regimes	39
36	Common implementation mistakes	40
37	Common misconceptions and the practical mental model	41
	References and further reading	42

The FlashAttention family evolves with GPU hardware. Chapters 1–20 explain the durable algorithmic ideas. Chapters 21–31 isolate generation-specific and framework-specific details so that the core mental model does not depend on one CUDA release or one GPU generation.

1 What FlashAttention actually optimizes

Start with the ordinary attention function:

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + B\right)V.$$

A textbook implementation often makes this look like three large operations: form the score matrix S, apply row-wise softmax to obtain P, then multiply P by V. Mathematically that is fine. On a GPU, however, writing a huge intermediate matrix to HBM and reading it back can be far more expensive than the equation suggests.

FlashAttention's central contribution is to make the algorithm **IO-aware**. It partitions the computation into tiles that fit in fast on-chip memory, streams blocks of K and V, and maintains enough row-wise softmax state to produce the exact output without materializing the full $N \times N$ attention matrix in HBM.[4]

> **What changes:** the execution schedule, memory traffic, and stored intermediates.
> **What does not change:** the dense scaled-dot-product attention function being evaluated.

This distinction is why "FlashAttention is a faster kind of attention" can be misleading. It is better thought of as an algorithm and kernel family for evaluating attention efficiently on accelerators. A model can use causal masking, RoPE, MQA/GQA, or other attention features and still use a FlashAttention implementation underneath.

The original paper contrasts this approach with approximate attention methods that reduce arithmetic by changing the mathematical problem. Dense FlashAttention does not make that trade. The same paper also introduced a *block-sparse* extension, but that sparse extension is a different case because omitting blocks changes which interactions are computed.[4]

2 The attention equation is not the implementation

Two implementations can compute the same mathematical function while moving very different amounts of data.

A simple materializing implementation may conceptually do:

$$S \leftarrow QK^\top/\sqrt{d}, \qquad P \leftarrow \mathrm{softmax}(S), \qquad O \leftarrow PV.$$

This presentation hides where tensors live. If S is written to HBM after the first matrix multiplication, read for softmax, P is written back, and then P is read again for the PV multiplication, the GPU spends significant time moving an N^2 object through memory.

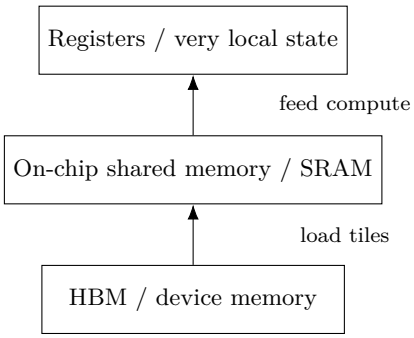
FlashAttention reorganizes the same dependencies. Blocks of Q, K, and V are brought near the compute units, score tiles are produced and consumed locally, and only compact row-wise statistics plus the output need to persist across tiles.[4, 5]

> **Algorithmic lesson.** A computational graph is not a memory schedule. Writing $P = \mathrm{softmax}(QK^\top)$ on paper does not require an implementation to store all of $QK^\top$ or P in off-chip memory at once.

This idea generalizes beyond attention. Fused kernels, tiling, recomputation, and operator scheduling often trade a small amount of extra arithmetic for much less movement of large intermediates. On modern accelerators, that can be the right trade because matrix-multiply throughput has grown much faster than many other parts of the memory and execution hierarchy. FlashAttention-3 and -4 make that hardware dependence increasingly explicit.[6, 7]

3 GPU memory hierarchy and why IO matters

GPUs expose a hierarchy rather than one uniform pool of equally fast memory. The exact names and capacities vary by architecture, but a useful mental model is:



HBM is large, but data must travel to the compute units. On-chip storage is much smaller, but reuse there is much cheaper. The original FlashAttention analysis models this asymmetry using HBM and SRAM and explicitly optimizes the number of transfers between them.[4]

The purpose of tiling is not simply "make tensors smaller." It is to increase **reuse while a tile is resident on chip**. A block of Q can interact with multiple K/V blocks before its partial softmax/output state is written back. Conversely, K/V blocks can be streamed through query blocks according to the chosen schedule.

> A kernel becomes IO-aware when the placement and movement of data are part of the algorithm, rather than an afterthought left to a sequence of separately launched tensor operations.

Do not turn this into a universal slogan that attention is always "memory-bound." The bottleneck depends on sequence lengths, head dimension, dtype, mask pattern, GPU generation, forward vs. backward, and which kernel is running. FA3 and FA4 exist partly because, as hardware changed, the dominant bottlenecks changed too.[6, 7]

4 Why materializing S and P is expensive

The quadratic intermediate becomes concrete very quickly. Suppose a batch contains one sequence, with 32 attention heads, sequence length $N = 8192$, and a two-byte dtype such as FP16 or BF16. One dense tensor with shape

$$[1, 32, 8192, 8192]$$

contains 32×8192^2 elements. At two bytes per element, that is

$$32 \times 8192^2 \times 2 = 4,294,967,296 \text{ bytes} = 4 \text{ GiB}.$$

That is the size of *one* full score- or probability-like tensor for this example.

A naïve decomposition may produce arrays of this scale at multiple stages. This does not mean every modern framework keeps both S and P alive simultaneously, and compiler fusion can already avoid some traffic. The example illustrates the fundamental problem: a dense N^2 intermediate is large enough that repeatedly writing and rereading it can dominate memory use and bandwidth.

FlashAttention avoids storing the full matrix in HBM. It forms score tiles, applies the softmax update while the tile is on chip, immediately uses those probabilities to accumulate the corresponding contribution from V, then discards the tile.[4]

> **Important wording:** FlashAttention removes the need to materialize the full attention matrix as an off-chip intermediate. It does *not* remove the logical pairwise interactions required by dense attention.

This is also why the memory benefit is especially important during training, where naïve autograd would otherwise want large intermediates for the backward pass.

5 Dense arithmetic is still quadratic

For self-attention with N tokens and head dimension d, forming all query-key scores requires work proportional to

$$N^2 d.$$

Multiplying the probabilities by values adds another dense pairwise matrix multiplication of the same broad order. FlashAttention reorganizes these operations, but it does not stop evaluating the dense set of query-key interactions.[4, 5]

So three statements must not be confused:

- **Arithmetic complexity:** dense attention remains $O(N^2 d)$.
- **Large intermediate storage:** FlashAttention avoids an $O(N^2)$ materialized score/probability tensor in HBM.
- **HBM traffic:** the original paper proves a lower IO cost under its two-level memory model than standard materializing attention.[4]

This explains how a method can make much longer sequences practical without making long context "free." Doubling N still roughly quadruples the number of dense query-key pairs. FlashAttention mainly attacks the data-movement and memory-footprint side of that computation.

> If you want to reduce the number of query-key pairs themselves, you need a different mathematical structure: for example sparsity, a local pattern, or a different attention formulation. Those choices can change model behavior and are conceptually separate from dense FlashAttention.

When reporting speedups, always distinguish asymptotic arithmetic from measured runtime. A kernel can become several times faster at the same $O(N^2 d)$ complexity because the constant factors, occupancy, fusion, and memory traffic change dramatically.

6 Memory-efficient exact attention predates FlashAttention

It is historically inaccurate to say FlashAttention first discovered that exact attention can avoid quadratic memory.

Rabe and Staats showed before FlashAttention that attention need not require $O(N^2)$ memory with respect to sequence length. Their work gave exact memory-efficient algorithms while retaining quadratic time, and a practical accelerator implementation with subquadratic memory.[3]

FlashAttention's distinct contribution was to turn memory efficiency into an explicit **IO-aware GPU algorithm**: tile the computation around the accelerator memory hierarchy, analyze HBM accesses, fuse the relevant operations, and show substantial wall-clock gains.[4]

Online softmax also has an earlier lineage. Milakov and Gimelshein described an online recurrence that computes the classical stable softmax normalizer with fewer memory accesses. FlashAttention builds the same kind of running-max/running-normalizer idea into tiled attention, while also accumulating the value-weighted output.[2, 4]

A precise lineage is therefore:

- stable / online softmax provides a streaming normalization tool,
- earlier memory-efficient attention shows exact attention need not store N^2 state,
- FlashAttention co-designs tiling, softmax state, and GPU IO to make the approach fast in practice.

This distinction matters because "memory efficient" and "IO optimal for a hardware model" are not the same claim.

7 Tiling queries, keys, and values

Instead of forming all $N_q N_k$ scores at once, partition the matrices into blocks. For one query block Q_i and one key/value block (K_j, V_j), compute

$$S_{ij} = \frac{Q_i K_j^\top}{\sqrt{d}} + B_{ij}.$$

The score tile S_{ij} is small enough to be processed near the compute units. Its contribution is folded into running row statistics and an output accumulator, then the tile can be discarded.

Conceptually, for each query block:

1. load a tile of Q,
2. stream compatible K/V tiles,
3. compute one score tile,
4. update row-wise softmax state,
5. accumulate the corresponding V contribution,
6. move to the next tile without writing a global S or P matrix.

Actual kernels choose tile shapes and loop order based on on-chip capacity, head dimension, GPU generation, causal structure, and work partitioning. The original FlashAttention IO analysis uses tile sizes derived from SRAM capacity M.[4]

> Tiling alone is not enough. Matrix multiplication tiles compose naturally because sums can be accumulated. Softmax couples every score in a row through a shared maximum and denominator, so we need a way to merge blocks without seeing the full row at once.

That is the key mathematical trick in the next chapters.

8 Softmax is the difficult part of streaming

A numerically stable softmax for one row $x_1, \ldots, x_N$ uses

$$m = \max_j x_j, \qquad \ell = \sum_j e^{x_j - m}, \qquad p_j = \frac{e^{x_j - m}}{\ell}.$$

The subtraction by m prevents overflow from large positive logits. But it seems to create a streaming problem: how can an early block be normalized if a later block may contain a larger maximum?

The answer is to retain sufficient statistics that can be **rescaled** when the maximum changes. Suppose the running state after earlier elements is $(m_{\text{old}}, \ell_{\text{old}})$ and a new block has maximum m_b. Define

$$m_{\text{new}} = \max(m_{\text{old}}, m_b).$$

Every contribution accumulated under the old maximum can be converted to the new reference by multiplying it by

$$\alpha = e^{m_{\text{old}} - m_{\text{new}}}.$$

The new block is evaluated relative to the same m_{new}.

This is not an approximation. It is the identity

$$e^{x - m_{\text{old}}} e^{m_{\text{old}} - m_{\text{new}}} = e^{x - m_{\text{new}}}.$$

> The running maximum is a change of numerical reference point. When that reference changes, previously accumulated exponentials can be rescaled exactly in real arithmetic rather than recomputed from scratch.

The online-normalizer recurrence predates FlashAttention and provides the mathematical basis for streaming stable softmax.[2]

9 Online softmax from first principles

Process scalar logits $x_1, x_2, \ldots$ one at a time. Initialize

$$m_0 = -\infty, \qquad \ell_0 = 0.$$

After observing x_j, update

$$m_j = \max(m_{j-1}, x_j),$$
$$\ell_j = \ell_{j-1} e^{m_{j-1} - m_j} + e^{x_j - m_j}.$$

Milakov and Gimelshein show that this produces the same stable softmax normalizer while requiring fewer passes over the input than the conventional safe-softmax procedure.[2]

For attention, we also need the weighted value sum. Introduce an unnormalized accumulator

$$a = \sum_j e^{x_j - m} v_j.$$

When the maximum changes from m to m', rescale both ℓ and a by $e^{m - m'}$. Then add the new exponentials and value contributions under the new reference.

At the end,

$$o = \frac{a}{\ell}.$$

> For attention, x_j is not a fixed input vector stored in advance. Each block of logits is generated on demand from a matrix product $Q_i K_j^\top / \sqrt{d}$ plus mask/bias terms. The online recurrence lets the kernel consume that block immediately.

The same idea works row by row and block by block, which is what makes a tiled exact softmax-attention forward pass possible.

10 The blockwise merge recurrence

For one query row, suppose the running state after some key blocks is

$$(m, \ell, a),$$

where m is the maximum score seen so far, ℓ is the stable softmax denominator under that maximum, and $a \in \mathbb{R}^{d_v}$ is the unnormalized value accumulator.

For a new score block $s \in \mathbb{R}^b$ with matching values $V_b \in \mathbb{R}^{b \times d_v}$, let

$$m_b = \max(s), \qquad m' = \max(m, m_b),$$

$$\alpha = e^{m - m'}, \qquad p = e^{s - m'}.$$

Then update

$$\ell' = \alpha \ell + \sum_r p_r,$$

$$a' = \alpha a + p^\top V_b,$$

$$m \leftarrow m', \quad \ell \leftarrow \ell', \quad a \leftarrow a'.$$

Finally,

$$o = a / \ell.$$

For a block of query rows, m and ℓ become row-wise vectors and a becomes a matrix. Masks can be applied to the score tile before the exponentials, with masked positions contributing zero probability.

> The recurrence is the algebraic reason tile boundaries do not change the dense softmax result. A different tiling changes the order of floating-point operations, but not the intended real-arithmetic function.

FlashAttention's published algorithms express equivalent running-max / running-normalizer / output updates in block form.[4, 5]

11 A complete numerical example

Consider one already-scaled, unmasked attention row

$$s = [2, 1, 4, 3]$$

with two-dimensional values

$$v_1 = [1, 0],\ v_2 = [0, 1],\ v_3 = [2, 0],\ v_4 = [0, 2].$$

The global maximum is 4, so stable unnormalized weights are

$$[e^{-2}, e^{-3}, 1, e^{-1}] \approx [0.135335, 0.049787, 1, 0.367879].$$

Their sum is

$$\ell \approx 1.55300179,$$

and full softmax gives

$$O \approx [1.37497284,\ 0.50582424].$$

Now process two blocks. For $[2, 1]$:

$$m_1 = 2, \quad \ell_1 = 1 + e^{-1} = 1.36787944, \quad a_1 = [1, e^{-1}].$$

For the second block $[4, 3]$, the new maximum is 4, so

$$\alpha = e^{2-4} = e^{-2}.$$

Then

$$\ell_2 = \alpha\ell_1 + 1 + e^{-1} = 1.55300179,$$

$$a_2 = \alpha a_1 + [2, 2e^{-1}] \approx [2.13533528, 0.78554595].$$

Therefore

$$a_2/\ell_2 \approx [1.37497284, 0.50582424],$$

matching the full-row computation.

> The old block was not revisited. Its contribution was merely rescaled when a larger maximum appeared.

12 FlashAttention forward pass

A useful conceptual forward pass is:

1. Partition Q into query-row tiles and K, V into key/value tiles.
2. For each query tile, initialize row-wise running maxima, normalizers, and output accumulators.
3. Load a key/value tile and form the local score tile $Q_i K_j^\top / \sqrt{d}$.
4. Apply causal/local masks or additive biases that belong to this tile.
5. Compute the tile maximum, update the running maximum, and rescale previous state.
6. Exponentiate the current tile relative to the updated maximum.
7. Update the denominator and the value-weighted output accumulator.
8. Continue until every required key tile has contributed.
9. Normalize the accumulator row-wise and write the output.

The original FlashAttention algorithm chooses block sizes so the relevant tiles and state fit in on-chip SRAM, reducing trips to HBM.[4]

An educational implementation can reproduce the algebra in a few lines of PyTorch, but such code is *not* a high-performance FlashAttention kernel. Production implementations depend on GPU-specific tiling, thread/warp scheduling, asynchronous copies, tensor-core instructions, and other low-level details.

> The essential algorithmic idea is independent of one CUDA kernel: generate a score tile, consume it immediately through online softmax and V accumulation, and never materialize the complete score/probability matrix in HBM.

Minimal educational PyTorch

The code below streams K/V blocks and implements the recurrence directly. It omits batching, masks, dropout, query tiling, and every GPU optimization, so it is a correctness illustration rather than a FlashAttention kernel.

```
def tiled_attention(q, k, v, block=128):
    scale = 1 / math.sqrt(q.shape[-1])
    m = q.new_full((q.shape[0],), -float("inf"))
    l = q.new_zeros(q.shape[0])
    a = q.new_zeros((q.shape[0], v.shape[-1]))
    for j in range(0, k.shape[0], block):
        kj, vj = k[j:j+block], v[j:j+block]
        s = (q @ kj.T) * scale
        m_new = torch.maximum(m, s.max(dim=-1).values)
        alpha = torch.exp(m - m_new)
        p = torch.exp(s - m_new[:, None])
        a = a * alpha[:, None] + p @ vj
        l = l * alpha + p.sum(dim=-1)
        m = m_new
    return a / l[:, None]
```

Later FlashAttention generations keep this semantic structure while changing how work is scheduled on newer hardware.[5, 6, 7]

13 Why dense FlashAttention is exact

Dense FlashAttention is called exact because the target function remains

$$\text{softmax}(QK^\top/\sqrt{d} + B)V.$$

Tiling does not delete query-key pairs. Online softmax does not replace the exponential or normalization with another function. The block recurrence simply changes the order in which sufficient statistics are accumulated.[4]

Three caveats keep the word **exact** precise:

- **Floating point is finite precision.** Reordering additions and reductions can produce small numerical differences from another implementation. PyTorch explicitly warns that SDPA backends can differ because floating-point operations are fused and ordered differently.[10, 13]
- **Dropout is stochastic during training.** Comparing two runs bit-for-bit requires matching random behavior in addition to mathematical attention semantics.
- **Sparse variants are different.** The original FlashAttention paper also presents block-sparse FlashAttention, which omits blocks and is therefore an approximate/sparse variant relative to full dense attention.[4]

> "Exact" does not mean "bitwise identical to every reference kernel." It means the algorithm is not intentionally changing dense softmax attention to reduce the mathematical work.

This distinction matters when evaluating numerical tests. A sensible tolerance depends on dtype, accumulation order, sequence length, and backend rather than requiring binary identity.

14 IO complexity: what the theorem actually says

The original FlashAttention paper analyzes a two-level memory model with HBM and on-chip SRAM of size M. Under the paper's assumptions, including head dimension d and

$$d \leq M \leq Nd,$$

standard materializing attention requires

$$\Theta(Nd + N^2)$$

HBM accesses, whereas FlashAttention requires

$$\Theta\left(\frac{N^2 d^2}{M}\right)$$

HBM accesses. The paper also proves an optimality result over a range of SRAM sizes in this model.[4]

Several details matter:

- These are **IO-complexity results in a particular memory model**, not a universal byte count for every GPU.
- The quantity counts movement of scalar elements/words between the modeled memory levels, up to asymptotic factors.
- It is not the arithmetic complexity. Dense attention still performs $O(N^2 d)$ work.
- Increasing usable on-chip memory M enables more reuse and reduces modeled HBM traffic.

> A common incorrect summary is "FlashAttention reduces attention IO from $O(N^2)$ to $O(N)$." The linear quantity is the large *auxiliary memory footprint* with respect to sequence length, not the general HBM-access expression above.

Intuitively, more usable on-chip memory lets larger working tiles stay resident and be reused for more attention work before data must be reloaded from HBM. In the theorem's regime, that increased reuse is why the HBM-access bound decreases as M grows.

15 Memory complexity: linear auxiliary state, not linear compute

Why do FlashAttention papers often say memory becomes linear instead of quadratic?

The inputs and output already contain $O(Nd)$ elements. A materializing dense attention implementation additionally creates $O(N^2)$ score/probability state. FlashAttention avoids storing those full matrices and retains only tiled working state plus row-wise statistics. As a result, the extra memory associated with the attention operation scales linearly with sequence length rather than quadratically.[4, 5]

For training, the difference is especially important because a straightforward backward pass might otherwise save a full probability matrix P. FlashAttention instead saves compact information such as output and row-wise normalization statistics, then recomputes score/probability tiles in backward.[4, 5]

> **Linear memory does not imply linear runtime.** The number of dense query-key interactions is still quadratic in N.

Also avoid claiming that total model memory is $O(N)$. Other Transformer components consume activation memory, and autoregressive serving has KV-cache memory that grows with context length. FlashAttention is specifically changing how the attention computation manages its intermediates.

This is one reason FlashAttention and activation checkpointing can coexist: both trade recomputation for reduced stored state, but at different scopes of the training graph.

16 Causal masking and tile skipping

In causal self-attention, query position i may not attend to future key positions $j > i$. A materialized mask would be another $N \times N$ object, but an efficient tiled kernel can reason about the geometry of each tile.

For square self-attention:

- blocks strictly above the causal boundary are fully masked and need not contribute,
- blocks strictly below the boundary are fully valid,
- only blocks intersecting the diagonal require element-level causal masking.

This avoids computing many invalid tiles in a causal kernel. FlashAttention-2 explicitly exploits causal structure, while current implementations also define precise alignment rules for unequal query/key lengths.[5, 8]

> Mask semantics are an API detail that must not be guessed. For example, current PyTorch SDPA treats a Boolean `attn_mask` value of `True` as a position that *participates* in attention, while other PyTorch mask APIs use different conventions.[10]

Local/sliding-window attention can similarly skip tiles outside the permitted window. But once the model intentionally restricts which pairs are attended, the *model's attention pattern* is sparse/local. FlashAttention can be the kernel used to execute that pattern, but it is no longer the same mathematical problem as unrestricted dense attention.

17 Backward pass: recompute instead of store

A naïve training implementation can save $P = \mathrm{softmax}(S)$ for backward. FlashAttention avoids keeping that N^2 tensor in HBM. Instead, it stores compact row-wise normalization information and recomputes score/probability tiles when gradients are needed.[4, 5]

For

$$S = QK^\top/\sqrt{d}, \qquad P = \mathrm{softmax}(S), \qquad O = PV,$$

a useful row-wise identity is

$$D_i = \sum_r (dO_{ir})O_{ir}.$$

After recomputing a tile of P, the local derivatives can be expressed as

$$dV \mathrel{+}= P^\top dO,$$

$$dP = dO\,V^\top,$$

$$dS = P \odot (dP - D[:,\mathrm{None}]),$$

then

$$dQ \mathrel{+}= dS\,K/\sqrt{d}, \qquad dK \mathrel{+}= dS^\top Q/\sqrt{d}.$$

Masks imply zero probability/gradient contribution for masked entries.

FA2 stores a row-wise log-sum-exp quantity that allows the probability tile to be reconstructed stably from recomputed scores.[5]

> Backward recomputation is intentional. It spends extra matrix-multiply work to avoid reading and writing a giant probability tensor, which can be a favorable trade on GPUs.

18 Why more FLOPs can still be faster

It is tempting to assume that fewer arithmetic operations always imply lower latency. Accelerator performance breaks that intuition regularly.

Matrix multiplication maps exceptionally well to tensor cores. HBM traffic, synchronization, shared-memory traffic, exponentials, reductions, and kernel launch boundaries can be comparatively expensive. FlashAttention therefore makes trades such as recomputing tiles in backward or doing rescaling arithmetic if that prevents much larger memory transfers.[4, 5]

FA2 makes this contrast explicit: one of its goals is to reduce **non-matmul FLOPs**, because those operations do not enjoy the same throughput as tensor-core GEMMs. The paper also improves work partitioning so more of the GPU is occupied.[5]

FA3 goes further on Hopper by overlapping matrix multiplication, softmax, and data movement using asynchronous hardware features. FA4 responds to Blackwell, where tensor-core throughput increased faster than some other resources, making exponentials and shared-memory traffic relatively more important.[6, 7]

> A better performance question is not merely "How many FLOPs?" but "Which operations, on which units, with what data movement, reuse, parallelism, and synchronization?"

This is the systems lesson that makes FlashAttention important beyond attention itself: hardware efficiency often comes from co-designing mathematical scheduling with the memory/execution hierarchy.

19 MHA, MQA, and GQA compatibility

FlashAttention does not require every architecture to have the same number of query and key/value heads.

Let

$$Q \in \mathbb{R}^{B \times N_q \times H_q \times d}, \qquad K, V \in \mathbb{R}^{B \times N_k \times H_{kv} \times d}.$$

For ordinary MHA, $H_q = H_{kv}$. In MQA, $H_{kv} = 1$. In GQA, $1 < H_{kv} < H_q$. Current Dao-AILab kernels support MQA/GQA by passing fewer KV heads than query heads, with the requirement that the number of query heads be divisible by the number of KV heads.[8]

The kernel still evaluates attention between each query head and its assigned KV head. Head sharing changes the architecture and KV-memory footprint. FlashAttention changes how the resulting attention operation is executed.

Orthogonal concepts:

- MQA/GQA: how heads share K/V projections,
- RoPE: how position transforms Q/K,
- FlashAttention: how attention is scheduled and computed efficiently.

These can be used together.

Current PyTorch SDPA also exposes `enable_gqa`. Its documentation labels GQA support experimental and imposes backend- and tensor-shape constraints, so production code should follow the exact version's documentation rather than assuming universal fused-kernel support.[10]

20 Variable lengths, local attention, and dropout

Production attention rarely consists only of equal-length dense sequences with no dropout. Current FlashAttention implementations support a broader feature set, but these are **implementation capabilities**, not properties of the mathematical idea itself.

The current Dao-AILab repository documents kernels/interfaces for features including:

- variable-length sequences,
- causal attention,
- local/sliding-window attention,
- dropout in training-oriented interfaces,
- MQA/GQA,
- ALiBi-style score bias in relevant interfaces,
- specialized incremental-decoding paths with KV cache, including optional RoPE handling.[8]

Feature support differs by CUDA/ROCm backend and evolves over time. For example, the repository documents separate NVIDIA and AMD backends with different implementation details and support matrices.[8]

Dropout deserves a practical warning. Current PyTorch `scaled_dot_product_attention` always applies dropout according to its `dropout_p` argument, so callers must pass 0.0 during evaluation when dropout should be disabled.[10]

> Do not infer feature support from the name "FlashAttention." Check the exact library, kernel generation, device, dtype, head dimension, mask/bias, and training/inference path that will actually run.

21 FlashAttention-2: what changed

FlashAttention-2 did not change the fundamental goal. It still computes exact attention while avoiding quadratic materialized intermediates. The improvement was mostly about using the GPU better.[5]

The FA2 paper identifies three main changes:

1. reduce the number of non-matmul FLOPs,
2. parallelize attention across sequence tiles even for a single head to improve occupancy,
3. repartition work among warps within a thread block to reduce shared-memory communication.[5]

In the paper's A100 experiments, these changes produced roughly a 2× speedup over FlashAttention-1 and reached 50–73% of theoretical maximum FLOPs/s, while end-to-end GPT-style training reached up to 225 TFLOPs/s per A100 under the reported setup.[5]

Those numbers are not universal speed guarantees. They depend on GPU, shapes, dtype, causal mode, software versions, and benchmark methodology.

> FA1's breakthrough was IO-aware tiled exact attention. FA2's breakthrough was better parallelism and work partitioning around the same high-level algorithmic idea.

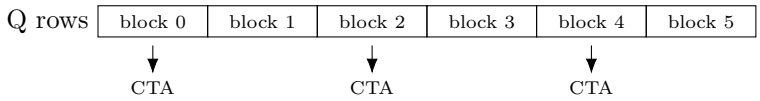
This pattern will repeat: later generations keep the semantics but adapt the kernel pipeline to the resources and bottlenecks of newer GPU architectures.

22 FA2 parallelism across sequence tiles

A GPU needs enough independent work to keep many streaming multiprocessors busy. In FA1, a natural unit of parallelism was roughly batch $\times$ head. If batch size and head count were small, that could leave hardware under-occupied even when the sequence itself was long.

FA2 adds parallelism along the sequence dimension. Different thread blocks can work on different query-row tiles for the same attention head. Because query rows are independent until their own key/value reductions are complete, this exposes more parallel work without changing attention semantics.[5]

A simplified mental picture is:



The important point is not the exact block assignment, which is kernel-specific. It is that long sequence length itself becomes a source of parallelism.

> More parallel blocks are not automatically better. Tile size, occupancy, registers, shared memory, and data reuse interact. Kernel design is an optimization problem over hardware resources, not just a request for maximum thread count.

23 FA2 work partitioning and non-matmul FLOPs

FA2 also changes work inside each thread block. FA1's warp partitioning could require partial results to be written to shared memory and combined, creating communication overhead. FA2 repartitions work so warps cooperate in a way that reduces shared-memory reads/writes.[5]

The paper also reduces operations that are expensive relative to tensor-core matrix multiplication. This includes algebraic restructuring around softmax rescaling and backward computations so fewer non-matmul FLOPs are performed.[5]

This matters because "one FLOP" is not a useful performance currency across all operation types. Tensor cores can execute matrix multiply-accumulate at enormous throughput, while exponentials, scalar reductions, conversions, and shared-memory synchronization have different limits.

> FA2 is a reminder that after IO is improved, the next bottleneck may be occupancy, warp communication, or non-matmul work. Optimization moves the bottleneck rather than making hardware constraints disappear.

FA2 also refines what is saved for backward, using compact row-wise log-sum-exp information and recomputing tiles instead of materializing probabilities. This preserves the linear-memory character while improving the kernel's work distribution.[5]

24 FlashAttention-3: the Hopper generation

Hopper GPUs introduced hardware features that changed the best way to schedule attention. FlashAttention-3 was designed specifically to exploit those capabilities.[6]

The FA3 paper points out that FA2 achieved only about 35% utilization on H100 in its measurements, leaving substantial headroom. It develops three broad ideas:

1. overlap computation and data movement using asynchrony and warp specialization,
2. interleave matrix multiplication and softmax work so different execution units remain busy,
3. exploit FP8 hardware with quantization techniques designed to control numerical error.[6]

On H100, the final NeurIPS 2024 publication reports 1.5-2.0x speedup over FA2 in its benchmark suite, BF16 throughput up to 840 TFLOPs/s (85% utilization), and FP8 throughput reaching 1.3 PFLOPs/s.[6]

Again, these are hardware- and benchmark-specific empirical results, not promises for arbitrary models.

> FA3 is not "a more approximate FlashAttention." The FP16/BF16 path preserves the exact-attention algorithmic goal. The FP8 path intentionally introduces lower-precision arithmetic and therefore needs its own numerical-accuracy discussion.

25 FA3 asynchrony: overlap data movement, GEMM, and softmax

On Hopper, Tensor Memory Accelerator (TMA) and asynchronous warp-group matrix-multiply operations make it possible to build a deeper software pipeline. FA3 uses warp specialization so different warps take responsibility for different stages, allowing data movement and computation to overlap.[6]

A simplified pipeline is:

$$\text{load next tiles} \quad \| \quad QK^\top \text{ GEMM} \quad \| \quad \text{softmax/update} \quad \| \quad PV \text{ GEMM}.$$

The symbol $\|$ means the stages are intentionally overlapped where dependencies permit, not that they are mathematically independent.

FA3 also uses a ping-pong style schedule to interleave block matrix multiplication and softmax. The point is to avoid leaving tensor cores idle while scalar/special-function work is performed, and vice versa.[6]

> The online-softmax recurrence is still present conceptually. What changes is the hardware pipeline used to execute the score, normalization, and value-accumulation stages.

This is an important progression in the family: FA1 asks "How do we avoid HBM traffic?", FA2 asks "How do we partition the work better?", and FA3 asks "How do we overlap distinct execution resources on Hopper?"

26 FA3 FP8: performance without pretending precision is free

Hopper provides very high tensor-core throughput for FP8. Simply converting all attention operands to FP8, however, can produce unacceptable numerical error because attention combines dot products, exponentials, normalization, and value accumulation over dynamic ranges that vary by tile.

FA3 introduces an FP8 path using block quantization and an incoherent processing technique to reduce quantization error while exploiting FP8 tensor-core throughput. In the final NeurIPS 2024 publication, FP8 FA3 reaches 1.3 PFLOPs/s on H100 and reports 2.6x lower numerical error than the baseline FP8 attention method used for comparison.[6]

> Do not conflate two meanings of "exact." FlashAttention's tiling/online-softmax algorithm can be exact with respect to the dense attention formula, while performing that formula in a lower-precision numeric format still introduces quantization/rounding error.

A model's acceptable precision depends on the full training/inference setup. FP8 support is therefore a hardware-and-numerics feature layered onto the FlashAttention execution strategy, not proof that FP8 is universally interchangeable with BF16/FP16.

The official repository continues to separate FA3's Hopper-specific implementation from other backends, which is another reason not to state FA3 capabilities as generic properties of "FlashAttention."[8]

27 FlashAttention-4: the Blackwell generation

FlashAttention-4 continues the same pattern of hardware-aware redesign, this time for NVIDIA Blackwell. The paper argues that Blackwell exhibits **asymmetric hardware scaling**: tensor-core throughput roughly doubled relative to Hopper while some other resources, including shared-memory bandwidth and exponential throughput, did not scale at the same rate.[7]

When one subsystem becomes much faster, operations that used to be secondary can become the bottleneck. FA4 therefore redesigns both forward and backward pipelines rather than merely reusing the Hopper schedule.

The paper's major techniques include:

- fully asynchronous MMA pipelines and larger tiles,
- software-emulated exponential plus conditional online-softmax rescaling in forward,
- Tensor Memory and 2-CTA MMA techniques to reduce shared-memory traffic and atomic additions in backward.[7]

On B200 with BF16, the paper reports up to 1.3× speedup over cuDNN 9.13 and 2.7× over its Triton comparison, reaching up to 1613 TFLOPs/s (71% utilization) under the evaluated configurations.[7]

> FA4 does not change the big-O arithmetic of dense attention. It attacks the new bottlenecks created by a GPU generation whose compute/memory/function-unit balance differs from Hopper.

28 FA4 and asymmetric hardware scaling

The FlashAttention lineage is a case study in why kernels cannot be optimized once and assumed optimal forever.

On Blackwell, matrix multiplication became fast enough that the forward pass can become constrained by softmax/exponential work relative to tensor-core GEMM. FA4 uses software-emulated exponential and avoids unnecessary rescaling when the running maximum does not require it, reducing pressure on slower non-matmul resources.[7]

The backward pass has a different bottleneck. FA4 uses Blackwell's Tensor Memory (TMEM) and 2-CTA MMA mode to reduce shared-memory traffic and atomic accumulation overhead.[7]

This yields an important systems principle:

> **A kernel is balanced against a particular hardware ratio.** If tensor-core throughput improves faster than memory bandwidth or special-function throughput, the optimal algorithmic pipeline can change even when the mathematical function is identical.

That is why "FlashAttention-4 is just a faster FA3" misses the interesting part. The algorithm and the kernel pipeline are co-designed around new asymmetries. The FlashAttention-4 title itself makes that point explicit.[7]

This also explains why performance numbers should always name the GPU generation. A speedup on B200 is not evidence for the same ratio on H100, A100, or a non-NVIDIA accelerator.

29 FA4 implementation and current status

The current official Dao-AILab repository exposes FlashAttention-4 as a CuTeDSL implementation optimized for Hopper and Blackwell GPUs such as H100 and B200, with installation through the `flash-attn-4` package.[8]

The FlashAttention-4 paper emphasizes another engineering change: the implementation is written in CuTe-DSL embedded in Python. The paper reports 20–30× faster compile times than the traditional C++ template-based approach used in its comparison while retaining the required low-level expressivity.[7]

As of September 2026, PyPI classifies flash-attn-4 as

`Development Status :: 3 - Alpha.`[9]

> Treat this as a current implementation snapshot, not an evergreen property. "FA4 exists in the official repository" and "every production environment should replace FA2 with FA4" are very different claims.

Hardware, CUDA, PyTorch, feature, and packaging requirements evolve quickly. The durable lesson is the algorithmic progression, while exact installation/support claims should always be checked against the current official repository before deployment.

30 FlashAttention-1 through -4 compared

	Main target	Core kernel/algorithm emphasis	Selected paper-reported results
FA1	IO / HBM traffic	Tiling, online softmax, fused exact attention, IO analysis	NeurIPS 2022 paper reports substantial training speedups vs. then-current baselines.[4]
FA2	GPU utilization	More sequence-level parallelism, fewer non-matmul FLOPs, improved warp work partitioning	About 2× over FA1 and 50–73% theoretical max FLOPs/s on A100 in paper.[5]
FA3	Hopper	Warp specialization, asynchronous TMA/WGMMA pipeline, GEMM-softmax overlap, FP8 path	1.5-2.0x over FA2 on H100 in paper, BF16 up to 840 TFLOPs/s.[6]
FA4	Blackwell-era asymmetry	Fully async MMA pipelines, larger tiles, software exp / conditional rescale, TMEM and 2-CTA backward techniques	Up to 1613 TFLOPs/s BF16 on B200 and 1.3× vs. cuDNN 9.13 in paper.[7]

The version number should not be interpreted as a sequence of different attention definitions. The family preserves the central goal of efficient attention evaluation while adapting scheduling and numerical paths to newer hardware.

The exact feature matrix differs across implementations. For example, FA3 is strongly associated with Hopper-specific optimization, while the current FA4 repository targets both Hopper and Blackwell through its CuTeDSL path.[8]

31 PyTorch scaled-dot-product attention today

The high-level PyTorch API is

```
torch.nn.functional.scaled_dot_product_attention(
    query, key, value,
    attn_mask=None,
    dropout_p=0.0,
    is_causal=False,
    scale=None,
    enable_gqa=False,
)
```

Current PyTorch documentation says CUDA SDPA can automatically select optimized implementations based on the inputs. The main API documentation explicitly discusses FlashAttention-2, a memory-efficient attention implementation, and the C++ math implementation. The preferred mechanism for restricting the backend is `torch.nn.attention.sdpa_kernel`.[10, 11]

For example:

```
from torch.nn.attention import SDPBackend, sdpa_kernel
import torch.nn.functional as F

with sdpa_kernel(SDPBackend.FLASH_ATTENTION):
    y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

If a requested fused kernel cannot run for the supplied inputs, PyTorch can warn with reasons when other fallbacks are disabled. Backend eligibility depends on device, dtype, shape, mask, and other arguments.[10]

Current `torch.nn.attention` documentation also exposes registration/activation hooks for newer FlashAttention implementations, including identifiers such as FA3 and FA4. This is a fast-moving integration surface, so use the installed PyTorch version's documentation as authority.[12]

32 Exactness is not bitwise identity

Suppose a reference math kernel and a FlashAttention kernel both implement the same dense SDPA definition. You should still expect small numerical differences.

Floating-point addition is not associative:

$$(a + b) + c \neq a + (b + c)$$

in finite precision for some values. Tiling changes reduction order. Fusion can change when values are rounded. Accumulation precision can differ. PyTorch's reproducibility documentation explicitly notes that SDPA backends can produce different results because they perform floating-point accumulation in different orders.[13]

Therefore numerical validation should use appropriate tolerances rather than bitwise equality. The expected tolerance depends on:

- FP32 vs. BF16/FP16/FP8,
- accumulation precision,
- sequence length and score distribution,
- hardware/backend,
- dropout/randomness.

Exact algorithm means no deliberate mathematical approximation of dense attention.
Bitwise identical implementation means every finite-precision operation lands on exactly the same bits. FlashAttention promises the first idea, not the second across arbitrary kernels.

Determinism is another separate question. Current libraries expose backend- and version-specific controls for deterministic behavior, which can carry performance or memory costs.[8, 13]

33 FlashAttention vs. PagedAttention

The names sound related, but they solve different problems.

	FlashAttention	PagedAttention
Primary problem	Efficiently execute attention, especially reducing large intermediate memory traffic	Manage dynamically growing KV-cache memory for LLM serving
Mechanism	Tiling, online softmax, fused/hardware-aware kernels	Paging-inspired KV-block allocation and mapping
Main state	Attention tiles and row-wise normalization/output state	Persistent per-request KV cache
Changes dense attention formula?	No for dense FlashAttention	No, primarily changes cache memory management

PagedAttention was introduced with vLLM to reduce KV-cache waste from fragmentation and duplication in high-throughput serving.[14]

FlashAttention is about how an attention operation consumes Q/K/V. PagedAttention is about how serving systems lay out and access persistent KV state across requests. They can coexist in the same serving stack.

The current Dao-AILab repository even exposes a KV-cache-oriented FlashAttention interface with optional block tables, illustrating that kernel execution and cache paging are composable concerns rather than mutually exclusive ideas.[8]

> If the problem statement contains "N^2 score/probability intermediates," think FlashAttention. If it contains "fragmented per-request KV-cache allocation," think PagedAttention.

34 FlashAttention vs. sparse and linear attention

Three very different strategies are often grouped together because all can make attention cheaper in some regime.

Dense FlashAttention keeps the dense softmax-attention function and changes how it is executed.

Sparse/local attention deliberately computes only a subset of query-key interactions. If each query attends to only a window or selected blocks, arithmetic can fall below full dense N^2 work, but the model's attention pattern has changed.

Linear-attention families change the mathematical formulation, often using feature maps or associativity so attention can be rearranged without an explicit dense softmax score matrix. Their semantics and approximation/exactness properties depend on the specific method.

> The phrase "memory-efficient attention" is too broad to identify a method. Ask two questions:
> 1. Does it compute the same dense softmax attention?
> 2. Does it reduce arithmetic pairs, or mostly memory traffic/intermediate storage?

The original FlashAttention paper explicitly contrasted its dense exact algorithm with approximate methods, and separately explored block-sparse FlashAttention as an approximate/sparse extension.[4]

This is why "FlashAttention makes attention linear" is wrong. If a system shows near-linear scaling because it uses a local window or another sparse pattern, the sparsity is what changed the number of interactions. FlashAttention may still be the kernel underneath.

35 Training, prefill, and decode are different regimes

FlashAttention is most naturally powerful when both query and key sequence dimensions are substantial, as in training or long-prompt prefill. There are many query rows, so avoiding the materialized attention matrix and exploiting tiled GEMMs provide large benefits.

Autoregressive decode is different. For a single new token, N_q may be one while N_k is the accumulated context length. The operation reads a large KV cache but produces only one new query row per sequence. In that regime, KV-cache bandwidth and serving/batching behavior can dominate, and specialized decode kernels matter. The official repository exposes `flash_attn_with_kvcache`, which can update and attend to the cache in one kernel and supports features such as MQA/GQA and optional RoPE handling.[8]

> This does not mean FlashAttention is irrelevant to inference. Prefill is an attention-heavy dense regime, and decode can still use specialized kernels from the same implementation family. It means "FlashAttention speeds up LLM inference" is incomplete unless you say which phase and bottleneck.

FlashAttention also cannot eliminate other costs: MLP layers, communication in tensor parallelism, KV-cache capacity, model-weight bandwidth, sampling, and scheduler overhead remain separate concerns.

36 Common implementation mistakes

1. **Assuming the flash backend always ran.** High-level frameworks may fall back when dtype, device, shape, or mask is unsupported. Use backend controls/profiling when it matters.[10]
2. **Leaving dropout on during evaluation.** PyTorch SDPA applies `dropout_p` exactly as supplied, so pass 0.0 when evaluation requires no dropout.[10]
3. **Using the wrong mask convention.** Boolean mask semantics differ across APIs.
4. **Expecting bitwise agreement.** Fused kernels can reorder floating-point operations.[13]
5. **Ignoring GQA head constraints.** Current interfaces require compatible/divisible query and KV head counts.[8, 10]
6. **Treating current head-dimension/dtype support as timeless.** These constraints change across releases and backends.[8]
7. **Benchmarking without synchronization/warmup.** Asynchronous GPU execution can make naïve wall-clock measurements meaningless.
8. **Calling a sparse/local pattern "dense FlashAttention."** Kernel and attention pattern are separate choices.
9. **Confusing prefill with decode.** The shapes and bottlenecks are different.
10. **Assuming newest generation is always deployable.** Current FA4 package metadata is alpha, and hardware/software compatibility must be checked.[9]

37 Common misconceptions and the practical mental model

"FlashAttention approximates softmax."
No. Dense FlashAttention evaluates the dense softmax-attention function using tiled online normalization.[4]

"FlashAttention makes attention $O(N)$."
No. Dense arithmetic remains $O(N^2 d)$. The large auxiliary attention-memory footprint becomes linear in sequence length, and HBM traffic is reduced.

"It is just kernel fusion."
Fusion is part of the implementation story, but the online-softmax tiling and IO-aware scheduling are algorithmic, not merely concatenating existing kernels.

"It is the same as PagedAttention."
No. PagedAttention manages KV-cache allocation for serving.[14]

"Exact means bitwise identical."
No. Floating-point operation order can change rounding.[13]

"Long context becomes free."
No. Dense pairwise arithmetic is still quadratic, and KV-cache / other model costs remain.

"FA1, FA2, FA3, and FA4 are different attention architectures."
No. They are successive generations of efficient kernels/algorithms shaped by different hardware bottlenecks.[4, 5, 6, 7]

Practical mental model: FlashAttention keeps score tiles close to the compute units, carries only the row statistics needed to merge softmax blocks exactly, consumes each probability tile immediately into the V accumulation, and avoids round-tripping a full N^2 attention matrix through HBM.

References and further reading

References

[1] Vaswani, A. et al. *Attention Is All You Need.* NeurIPS 2017. https://arxiv.org/abs/1706.03762

[2] Milakov, M., Gimelshein, N. *Online normalizer calculation for softmax.* arXiv:1805.02867, 2018. https://arxiv.org/abs/1805.02867

[3] Rabe, M. N., Staats, C. *Self-attention Does Not Need $O(n^2)$ Memory.* arXiv:2112.05682, 2021/2022. https://arxiv.org/abs/2112.05682

[4] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. https://arxiv.org/abs/2205.14135

[5] Dao, T. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.* ICLR 2024. https://arxiv.org/abs/2307.08691

[6] Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., Dao, T. *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision.* NeurIPS 2024. DOI: 10.52202/079017-2193.

[7] Zadouri, T., Hoehnerbach, M., Shah, J., Liu, T., Thakkar, V., Dao, T. *FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling.* arXiv:2603.05451, 2026. https://arxiv.org/abs/2603.05451

[8] Dao-AILab. *flash-attention official repository and current implementation documentation.* https://github.com/Dao-AILab/flash-attention

[9] PyPI. flash-attn-4 package metadata and release history; package classifier "Development Status :: 3 - Alpha." https://pypi.org/project/flash-attn-4/

[10] PyTorch. *torch.nn.functional.scaled_dot_product_attention documentation.* https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html

[11] PyTorch. *torch.nn.attention.sdpa_kernel documentation.* https://docs.pytorch.org/docs/stable/generated/torch.nn.attention.sdpa_kernel.html

[12] PyTorch. *torch.nn.attention module and FlashAttention implementation registration APIs.* https://docs.pytorch.org/docs/stable/nn.attention.html

[13] PyTorch. *Reproducibility documentation, including scaled-dot-product attention backend differences.* https://docs.pytorch.org/docs/stable/notes/randomness.html

[14] Kwon, W. et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* SOSP 2023. https://arxiv.org/abs/2309.06180

Reference map: what to read for which question

Question	Best starting source
What is the original IO-aware algorithm?	FlashAttention-1 paper [4]
Why does online normalization work?	Milakov & Gimelshein [2]
Was exact subquadratic-memory attention known earlier?	Rabe & Staats [3]
What specifically changed in FA2?	FlashAttention-2 paper [5]
What is Hopper-specific in FA3?	FlashAttention-3 paper [6]
What changes on Blackwell in FA4?	FlashAttention-4 paper [7]
What does the current official package support?	Dao-AILab repository [8]
How does current PyTorch select SDPA kernels?	PyTorch SDPA and `sdpa_kernel` docs [10, 11]
Why can exact backends differ numerically?	PyTorch reproducibility notes [13]
How is PagedAttention different?	Kwon et al. [14]

Implementation support changes faster than the mathematics. For deployed systems, treat the installed framework's documentation and the exact kernel repository/release as authoritative for supported devices, dtypes, head dimensions, masks, dropout, and GQA behavior.